



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

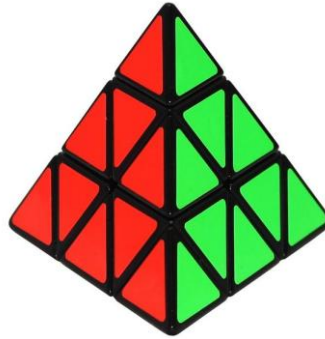
SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 05 de marzo del 2026

CALIFICACIÓN: _____

ACTIVIDAD

La actividad en esta practica consiste en generar una pirámide rubik (también conocida como **pyraminx**) de 9 pirámides por cara, cada una tendrá un color diferente, además de que se pide visualizar las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña).



PROPUESTA 1

Para esta primera propuesta se tomará el camino “fácil” esto significa instanciar un **total de 36 pirámides + un núcleo negro**, visto de otra manera es construir 4 caras “planas”, cada una conformada por 9 pirámides idénticas y un centro oscuro que nos ayudará a simular las separaciones negras del Rubik.

En esta propuesta cada mini pirámide se colorea mediante el color uniform (toda la pirámide de un mismo color) además de que esta propuesta es más simple y directa, pero no por ello la ideal.

El primer cambio lo hacemos en la función `void CrearPiramideTriangular()` donde modificamos los vértices, esto se hizo con valores ideales para que la pirámide fuera regular (aristas iguales). Así, al instanciarla muchas veces, todas las piezas encajan simétricamente sin deformaciones.

```
87 // -----
88 // V1 ----- CAMBIO 1: Modificamos los vertices de la piramide triangular
89 void CrearPiramideTriangular()
90 {
91     unsigned int indices_piramide_triangular[] = {
92         0, 1, 2, // Base
93         1, 3, 2, // Cara lateral derecha
94         3, 0, 2, // Cara lateral izquierda
95         1, 0, 3 // Cara frontal
96     };
97     GLfloat vertices_piramide_triangular[] = {
98         -0.5f, -0.2041f, 0.2886f, // 0: Base - Frente Izquierda
99         0.5f, -0.2041f, 0.2886f, // 1: Base - Frente Derecha
100        0.0f, -0.2041f, -0.5773f, // 2: Base - Atrás
101        0.0f, 0.6124f, 0.0f // 3: Punta superior
102    };
103    Mesh* obj1 = new Mesh();
104    obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
105    meshList.push_back(obj1);
106 }
107 // -----
```

Después dentro de nuestro main definimos los colores a usar para nuestras pirámides, todos estos con cierto grado de saturación.

```

336 // ----- CAMBIO 2/3:Se define la paleta de colores
337 // V1/2 ----- CAMBIO 2/3:Se define la paleta de colores
338 glm::vec3 cVerde = glm::vec3(0.141f, 0.310f, 0.227f);
339 glm::vec3 cRojo = glm::vec3(0.310f, 0.141f, 0.184f);
340 glm::vec3 cAzul = glm::vec3(0.141f, 0.204f, 0.310f);
341 glm::vec3 cAmarillo = glm::vec3(0.471f, 0.455f, 0.208f);
342 glm::vec3 cNegro = glm::vec3(0.110f, 0.102f, 0.102f); // Para las caras internas
343 // -----

```

Dentro de `while (!mainWindow.getShouldClose())` y después de llamar al shader 0 y `uniformColor` hacemos que todas las piezas que vayamos dibujando vaya saliendo de un solo color uniforme.

Empezamos creando una pirámide base (`modelBase`) y la alejamos 5 unidades de nosotros (`Z=-5`) además de aplicar las rotaciones en X,Y,Z. Después dibujamos nuestra pirámide negra de fondo para que las separaciones sean visibles.

```

416 // ===== VERSION 1 - CAMBIO 3 =====
417 // ===== VERSION 1 - CAMBIO 3 =====
418
419 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); //(Lente de la cámara)
420 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix())); //(Pos.Rot. d cámara)
421
422 // PASO 1. Matriz BASE para todo el Pyraminx (Posición y Rotaciones, SIN ESCALA)
423 glm::mat4 modelBase = glm::mat4(1.0f);
424 modelBase = glm::translate(modelBase, glm::vec3(0.0f, 0.0f, -5.0f));
425 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
426 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
427 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
428
429 // PASO 2: EL NÚCLEO NEGRO Para simular fondo
430 glm::mat4 modelCore = glm::scale(modelBase, glm::vec3(3.0f, 3.0f, 3.0f)); //Escala de 3
431 glUniform3fv(uniformColor, 1, glm::value_ptr(cNegro));
432 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCore));
433 meshList[1]->RenderMesh();

```

Definimos entonces las nueve posiciones donde vamos a pegar nuestras pequeñas pirámides por cara mediante un arreglo de posiciones, aquí definimos nuestros desplazamientos para X y Z. Los números dados no son otros que aproximaciones de un triángulo equilátero ($1/\sqrt{3}$, $0.5/\sqrt{3}$, $2/\sqrt{3}$). Pintamos toda la cara de color verde, y después orientamos este plano a la cara frontal de nuestro triángulo oscuro mediante funciones de rotación y traslación. Es importante señalar que se encuentra 0.07 unidades fuera de la pirámide negra.

```

435 // PASO 3: DIBUJANDO CARA FRONTAL (VERDE)
436 float posXZ[9][2] = {
437     { 0.0f, -1.1547f}, {-0.5f, -0.2887f}, { 0.5f, -0.2887f}, // 3 Arriba
438     {-1.0f,  0.5773f}, { 0.0f,  0.5773f}, { 1.0f,  0.5773f}, // 3 Abajo
439     { 0.0f, -0.5774f}, {-0.5f,  0.2887f}, { 0.5f,  0.2887f} // 3 Invertidas
440 };
441 glUniform3fv(uniformColor, 1, glm::value_ptr(cVerde));
442 // Usamos modelBase para que los triángulos conserven su tamaño 1.0 original
443 glm::mat4 modelCaraFrontal = modelBase;
444 modelCaraFrontal = glm::rotate(modelCaraFrontal, glm::radians(70.5288f), glm::vec3(1.0f, 0.0f, 0.0f));
445 modelCaraFrontal = glm::translate(modelCaraFrontal, glm::vec3(0.0f, 0.68f, 0.0f)); //0.61 es al ras

```

Este proceso se repite otras nueve veces mediante un ciclo for, donde vamos moviendo la pieza de su “celda” dentro de la cara que ya definimos mediante posXZ. Nuestra condicional if nos apoya en colocar nuestros 3 triángulos invertidos finales.

```

446 // Repetimos proceso 9 veces
447 for (int i = 0; i < 9; i++) {
448     glm::mat4 modelPieza = modelCaraFrontal;
449     modelPieza = glm::translate(modelPieza, glm::vec3(posXZ[i][0], 0.0f, posXZ[i][1]));
450     modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
451     modelPieza = glm::translate(modelPieza, glm::vec3(0.0f, 0.2041f, 0.0f));
452     if (i >= 6) {
453         modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
454     }
455     modelPieza = glm::scale(modelPieza, glm::vec3(0.92f, 0.92f, 0.92f));
456     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPieza));
457     meshList[1] -> RenderMesh();
458 }

```

Así terminamos la cara verde, repitiendo esto para las 3 caras restantes obteniendo así un total de 36 pirámides y un núcleo.

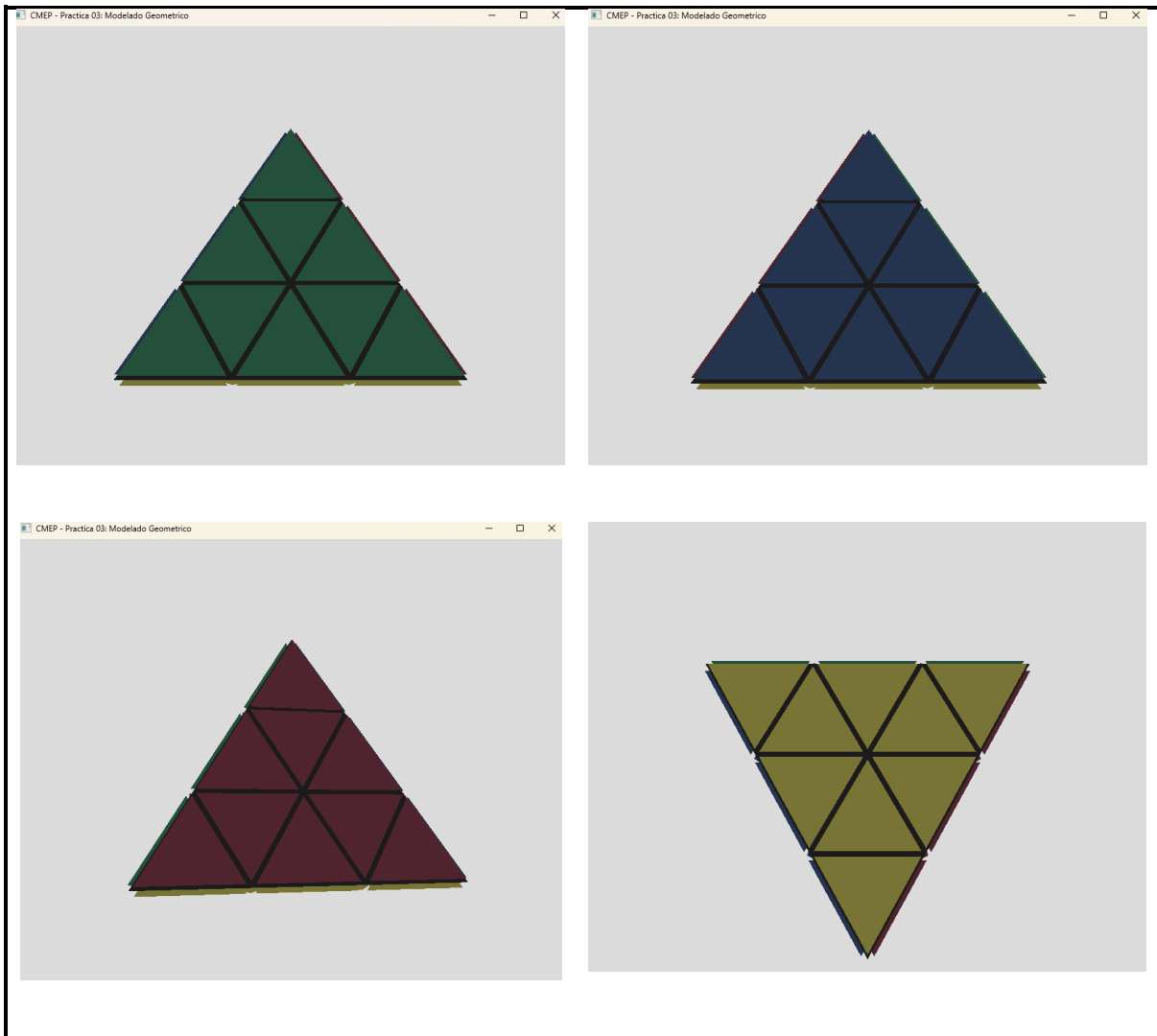
```

460 // PASO 4: DIBUJANDO CARA DERECHA (ROJA)
461 glUniform3fv(uniformColor, 1, glm::value_ptr(cRojo));
462 glm::mat4 modelCaraDerecha = modelBase;
463 // Rotamos la cara entera 120 grados para ponerla a un lado
464 modelCaraDerecha = glm::rotate(modelCaraDerecha, glm::radians(120.0f), glm::vec3(0.0f, 1.0f, 0.0f));
465 modelCaraDerecha = glm::rotate(modelCaraDerecha, glm::radians(70.5288f), glm::vec3(1.0f, 0.0f, 0.0f));
466 modelCaraDerecha = glm::translate(modelCaraDerecha, glm::vec3(0.0f, 0.68f, 0.0f));
467 for (int i = 0; i < 9; i++) {
468     glm::mat4 modelPieza = modelCaraDerecha;
469     modelPieza = glm::translate(modelPieza, glm::vec3(posXZ[i][0], 0.0f, posXZ[i][1]));
470     modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
471     modelPieza = glm::translate(modelPieza, glm::vec3(0.0f, 0.2041f, 0.0f));
472     if (i >= 6) {
473         modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
474     }
475     modelPieza = glm::scale(modelPieza, glm::vec3(0.92f, 0.92f, 0.92f));
476     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPieza));
477     meshList[1] -> RenderMesh();
478 }
479
480 // PASO 5: DIBUJANDO CARA IZQUIERDA (AZUL)
481 glUniform3fv(uniformColor, 1, glm::value_ptr(cAzul));
482 glm::mat4 modelCaraIzquierda = modelBase;
483 // 240 grados para posicionarla a la izquierda
484 modelCaraIzquierda = glm::rotate(modelCaraIzquierda, glm::radians(240.0f), glm::vec3(0.0f, 1.0f, 0.0f));
485 modelCaraIzquierda = glm::rotate(modelCaraIzquierda, glm::radians(70.5288f), glm::vec3(1.0f, 0.0f, 0.0f));
486 modelCaraIzquierda = glm::translate(modelCaraIzquierda, glm::vec3(0.0f, 0.68f, 0.0f));
487 for (int i = 0; i < 9; i++) {
488     glm::mat4 modelPieza = modelCaraIzquierda;
489     modelPieza = glm::translate(modelPieza, glm::vec3(posXZ[i][0], 0.0f, posXZ[i][1]));
490     modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
491     modelPieza = glm::translate(modelPieza, glm::vec3(0.0f, 0.2041f, 0.0f));
492     if (i >= 6) {
493         modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
494     }
495     modelPieza = glm::scale(modelPieza, glm::vec3(0.92f, 0.92f, 0.92f));
496     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPieza));
497     meshList[1] -> RenderMesh();
498 }
499
500 // PASO 6: DIBUJANDO CARA DE ABAJO (AMARILLA)
501 glUniform3fv(uniformColor, 1, glm::value_ptr(cAmarillo));
502 glm::mat4 modelCaraAbajo = modelBase;
503 // Giramos 180 grados sobre el eje Z.
504 modelCaraAbajo = glm::rotate(modelCaraAbajo, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
505 modelCaraAbajo = glm::translate(modelCaraAbajo, glm::vec3(0.0f, 0.68f, 0.0f));
506 for (int i = 0; i < 9; i++) {
507     glm::mat4 modelPieza = modelCaraAbajo;
508     modelPieza = glm::translate(modelPieza, glm::vec3(posXZ[i][0], 0.0f, posXZ[i][1]));
509     modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
510     modelPieza = glm::translate(modelPieza, glm::vec3(0.0f, 0.2041f, 0.0f));
511     if (i >= 6) {
512         modelPieza = glm::rotate(modelPieza, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
513     }
514     modelPieza = glm::scale(modelPieza, glm::vec3(0.92f, 0.92f, 0.92f));
515     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPieza));
516     meshList[1] -> RenderMesh();
517 }
518 // =====

```

EJECUCIÓN Versión 1

Podemos observar las pequeñas pirámides sobresaliendo del núcleo negro.



Se adjunta un link a un GIF para visualizar mejor el **pyraminx VERSION 1**

<https://drive.google.com/file/d/15fTDfUyXnIS6c4z9wYoDCA3DRJozQ8V0/view?usp=sharing>

PROPUESTA 2

La solución presentada fue la más sencilla, pero no la más “profesional” para esta segunda propuesta se busca instanciar menos pirámides. Tengamos en cuenta que un pyraminx real no tiene solo pegatinas sobre sus cuatro caras o no tiene 36 pirámides, su construcción es más compleja, pero a la vez con menos instancias de cada parte del Rubik. Para ello

crearemos un núcleo negro, 10 pirámides externas con caras de distinto color, y 12 rellenos (3 pirámides invertidas* 4 caras) dando así un total de solo **23 instancias**.

El primer cambio será agregar una lista para utilizar meshColor, ya que deberemos poner distintos colores en solo una pirámide.

```

33     vector<Mesh*> meshList;
34     vector<Shader>shaderList;
35     // -----
36     // V2 ----- CAMBIO 1: Agregamos el vector MeshColor
37     vector<MeshColor*> meshColorList;
38     // -----

```

Agregamos la función CrearPiezaPyraminx() donde construiremos cada pirámide con parámetros de color distintos, en vez de compartir vértices duplicamos los vértices por cara.

```

110     // -----
111     // V2 ----- CAMBIO 2: Función para crear piezas del Pyraminx con colores personalizados
112     void CrearPiezaPyraminx(glm::vec3 cFrente, glm::vec3 cDer, glm::vec3 cIzq, glm::vec3 cBase)
113     {
114         GLfloat vertices[] = {
115             // CARA FRONTAL (cFrente)
116             -0.5f, -0.2041f, 0.2886f, cFrente.r, cFrente.g, cFrente.b,
117             0.5f, -0.2041f, 0.2886f, cFrente.r, cFrente.g, cFrente.b,
118             0.0f, 0.6124f, 0.0f, cFrente.r, cFrente.g, cFrente.b,
119
120             // CARA DERECHA (cDer)
121             0.5f, -0.2041f, 0.2886f, cDer.r, cDer.g, cDer.b,
122             0.0f, -0.2041f, -0.5773f, cDer.r, cDer.g, cDer.b,
123             0.0f, 0.6124f, 0.0f, cDer.r, cDer.g, cDer.b,
124
125             // CARA IZQUIERDA (cIzq)
126             0.0f, -0.2041f, -0.5773f, cIzq.r, cIzq.g, cIzq.b,
127             -0.5f, -0.2041f, 0.2886f, cIzq.r, cIzq.g, cIzq.b,
128             0.0f, 0.6124f, 0.0f, cIzq.r, cIzq.g, cIzq.b,
129
130             // BASE (cBase)
131             -0.5f, -0.2041f, 0.2886f, cBase.r, cBase.g, cBase.b,
132             0.0f, -0.2041f, -0.5773f, cBase.r, cBase.g, cBase.b,
133             0.5f, -0.2041f, 0.2886f, cBase.r, cBase.g, cBase.b,
134         };
135
136         MeshColor* pieza = new MeshColor();
137         // 12 vértices * 6 datos (XYZ RGB) = 72 floats en total
138         pieza->CreateMeshColor(vertices, 72); // <--- CAMBIA ESTO A 72
139         meshColorList.push_back(pieza);
140     }
141     // -----

```

Además, nuevamente definimos la paleta de colores a pasar a esta función

```

336     // -----
337     // V1/2 ----- CAMBIO 2/3: Se define la paleta de colores
338     glm::vec3 cVerde = glm::vec3(0.141f, 0.310f, 0.227f);
339     glm::vec3 cRojo = glm::vec3(0.310f, 0.141f, 0.184f);
340     glm::vec3 cAzul = glm::vec3(0.141f, 0.204f, 0.310f);
341     glm::vec3 cAmarillo = glm::vec3(0.471f, 0.455f, 0.208f);
342     glm::vec3 cNegro = glm::vec3(0.110f, 0.102f, 0.102f); // Para las caras internas
343     // -----

```


Enseguida mandamos a llamar a nuestra función `CrearPiezaPyraminx()` pasando como parámetros las pirámides a construir con sus colores correspondientes, mencionando además que las caras no visibles de pondrán de color negro.

```

345 // ----- CAMBIO 4: Creación de las piezas del Pyraminx con colores personalizados. Se crean 14 piezas:
346 // 4 esquinas, 6 aristas y 4 centros
347 // GRUPO 1: LAS 4 ESQUINAS (PUNTAS): Orden de parámetros: Frente, Derecha, Izquierda, Base
348 CrearPiezaPyraminx(cVerde, cRojo, cAzul, cNegro); // 0. Punta Superior (Tiene Verde, Rojo y Azul)
349 CrearPiezaPyraminx(cVerde, cNegro, cAzul, cAmarillo); // 1. Esquina Inf Izq (Verde, Azul, Amarillo)
350 CrearPiezaPyraminx(cVerde, cRojo, cNegro, cAmarillo); // 2. Esquina Inf Der (Verde, Rojo, Amarillo)
351 CrearPiezaPyraminx(cNegro, cRojo, cAzul, cAmarillo); // 3. Esquina Trasera (Rojo, Azul, Amarillo)
352 // GRUPO 2: LAS 6 ARISTAS (PIEZAS DE BORDE) ---
353 CrearPiezaPyraminx(cVerde, cRojo, cNegro, cNegro); // 4. Arista Frontal-Derecha (Verde y Rojo)
354 CrearPiezaPyraminx(cVerde, cNegro, cAzul, cNegro); // 5. Arista Frontal-Izquierda (Verde y Azul)
355 CrearPiezaPyraminx(cNegro, cRojo, cAzul, cNegro); // 6. Arista Trasera (Rojo y Azul)
356 CrearPiezaPyraminx(cVerde, cNegro, cNegro, cAmarillo); // 7. Arista Base-Frontal (Verde y Amarillo)
357 CrearPiezaPyraminx(cNegro, cRojo, cNegro, cAmarillo); // 8. Arista Base-Derecha (Rojo y Amarillo)
358 CrearPiezaPyraminx(cNegro, cNegro, cAzul, cAmarillo); // 9. Arista Base-Izquierda (Azul y Amarillo)
359 // GRUPO 3: LOS 4 CENTROS (LA CARA VISIBLE SERÁ LA BASE) ---
360 CrearPiezaPyraminx(cNegro, cNegro, cNegro, cVerde); // 10. Centro Frente (El Verde va en la BASE)
361 CrearPiezaPyraminx(cNegro, cNegro, cNegro, cRojo); // 11. Centro Derecha (El Rojo va en la BASE)
362 CrearPiezaPyraminx(cNegro, cNegro, cNegro, cAzul); // 12. Centro Izquierda (El Azul va en la BASE)
363 CrearPiezaPyraminx(cNegro, cNegro, cNegro, cAmarillo); // 13. Centro Base (El Amarillo va en la BASE)
364 CrearPiezaPyraminx(cNegro, cNegro, cNegro, cNegro); // 14. NÚCLEO NEGRO SÓLIDO (Para rellenar el interior)
365
366 // -----

```

Finalmente, ya dentro de nuestro `while (!mainWindow.getShouldClose())` usamos ahora el shader 1, quien soporta color por vértice, y replicamos el paso inicial de nuestra propuesta 1 donde hacemos el modelo base, lo trasladamos más lejos de la pantalla y aplicamos rotaciones. Incluyendo que para facilitar el posicionamiento de las piezas definimos constantes para solo colocarlas en positivo o negativo.

```

527 shaderList[1].useShader();
528 uniformModel = shaderList[1].getModelLocation();
529 uniformProjection = shaderList[1].getProjectLocation();
530 uniformView = shaderList[1].getViewLocation();
531 // ===== VERSION 2 - CAMBIO 5 =====
532 // =====
533 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); //(Lente de la cámara)
534 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix())); //(Pos.Rot. d cámara)
535
536 // PASO 1. Matriz BASE para todo el Pyraminx (Posición y Rotaciones, SIN ESCALA)
537 glm::mat4 modelBase = glm::mat4(1.0f);
538 modelBase = glm::translate(modelBase, glm::vec3(0.0f, -0.5f, -4.0f));
539 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
540 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
541 modelBase = glm::rotate(modelBase, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
542
543 // PASO 2. Creación de constantes matemáticas de apoyo para posicionar las piezas.
544 // Las definimos así para evitar repetir cálculos trigonométricos en cada pieza
545 float H = 0.8165f;
546 float dZ1 = 0.2887f;
547 float dZ2 = 0.5774f;
548 float s = 0.90f;

```

Empezamos con el grupo 0, que contiene nuestro núcleo negro.

```

550 // PASO 3. DIBUJANDO PIEZAS DEL PYRAMINX
551 // ----- GRUPO 0: NÚCLEO NEG
552 glm::mat4 modelCore = modelBase;
553 // Ajuste exacto para alinear el centro del núcleo con el centro de las 14 piezas
554 modelCore = glm::translate(modelCore, glm::vec3(0.0f, -0.4083f, 0.0f));
555 // Escala de 2.85 (El Pyraminx total mide 3.0)
556 modelCore = glm::scale(modelCore, glm::vec3(2.85f, 2.85f, 2.85f));
557 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCore));
558 meshColorList[14] -> RenderMeshColor(); // Dibujamos la pieza 14 (Toda negra)

```

Después creamos nuestras 10 primeras piezas, correspondientes a las esquinas del pyraminx armando un arreglo de posiciones y, mediante un for, escalando cada pieza, trasladándola y dibujándola con meshColorList.

```

559 // ----- GRUPO 1: 10 PIEZAS VERTICALES (ESQUINAS Y ARISTAS)
560 glm::vec3 posiciones[10] = {
561     glm::vec3(0.0f, H, 0.0f), // 0: Punta Superior
562     glm::vec3(-1.0f, -H, dZ2), // 1: Esq Inf Izq
563     glm::vec3(1.0f, -H, dZ2), // 2: Esq Inf Der
564     glm::vec3(0.0f, -H, -1.1547f), // 3: Esq Trasera
565     glm::vec3(0.5f, 0.0f, dZ1), // 4: Arista Front-Der
566     glm::vec3(-0.5f, 0.0f, dZ1), // 5: Arista Front-Izq
567     glm::vec3(0.0f, 0.0f, -dZ2), // 6: Arista Trasera
568     glm::vec3(0.0f, -H, dZ2), // 7: Arista Base Front
569     glm::vec3(0.5f, -H, -dZ1), // 8: Arista Base Der
570     glm::vec3(-0.5f, -H, -dZ1) // 9: Arista Base Izq
571 };
572 for (int i = 0; i < 10; i++) {
573     glm::mat4 model = modelBase;
574     model = glm::translate(model, posiciones[i]);
575     model = glm::scale(model, glm::vec3(s, s, s)); // Escala desde el centro para el marco negro
576     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
577     meshColorList[i] -> RenderMeshColor();
578 }

```

El grupo 2A lo usamos para nuestras piezas de relleno de cada cara, frontal, derecha e izquierda, mientras que el 2B usa la misma lógica, pero con una rotación extra para las caras de relleno de nuestra cara inferior.

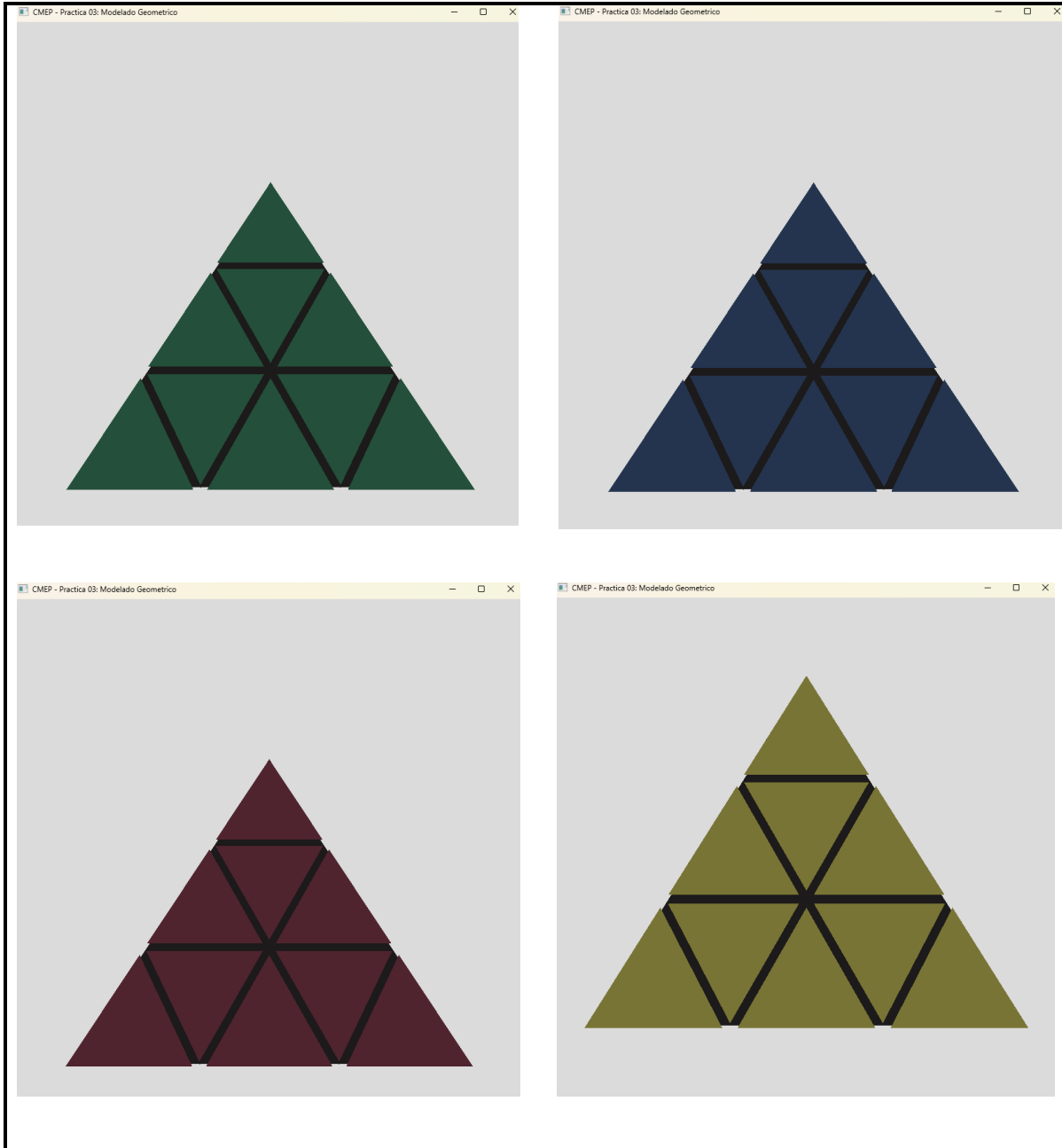
```

579 // ----- GRUPO 2: 12 PIEZAS DE RELLENO (HUECOS INVERTIDOS)
580 // A) HUECOS DE LAS 3 CARAS LATERALES
581 glm::vec3 huecosCara[3] = {
582     glm::vec3(0.0f, 0.2722f, 0.1925f), // Hueco Superior
583     glm::vec3(-0.5f, -0.5443f, 0.4811f), // Hueco Inferior Izquierdo
584     glm::vec3(0.5f, -0.5443f, 0.4811f) // Hueco Inferior Derecho
585 };
586 float angulosY[3] = { 0.0f, 120.0f, -120.0f }; // Frente, Derecha, Izquierda
587 int indicesColor[3] = { 10, 11, 12 }; // Reutilizamos tus piezas 10, 11 y 12
588 for (int cara = 0; cara < 3; cara++) {
589     for (int h = 0; h < 3; h++) {
590         glm::mat4 modelC = modelBase;
591         // Rotar la cara hacia su lado correspondiente
592         modelC = glm::rotate(modelC, glm::radians(angulosY[cara]), glm::vec3(0.0f, 1.0f, 0.0f));
593         modelC = glm::translate(modelC, huecosCara[h]);
594         // Acostar la pieza para que la base embone en la cara
595         modelC = glm::rotate(modelC, glm::radians(-109.471f), glm::vec3(1.0f, 0.0f, 0.0f));
596         modelC = glm::scale(modelC, glm::vec3(s, s, s));
597         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelC));
598         meshColorList[indicesColor[cara]] -> RenderMeshColor();
599     }
600 }
601 // B) HUECOS DE LA CARA BASE (Amarillo)
602 glm::vec3 huecosBase[3] = {
603     glm::vec3(-0.5f, -H, dZ1), // Hueco Izquierdo
604     glm::vec3(0.5f, -H, dZ1), // Hueco Derecho
605     glm::vec3(0.0f, -H, -dZ2) // Hueco Trasero
606 };
607 for (int h = 0; h < 3; h++) {
608     glm::mat4 modelC = modelBase;
609     modelC = glm::translate(modelC, huecosBase[h]);
610     // Rotar 180 en el eje Y para que los picos del triángulo encajen al revés
611     modelC = glm::rotate(modelC, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
612     modelC = glm::scale(modelC, glm::vec3(s, s, s));
613     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelC));
614     meshColorList[13] -> RenderMeshColor(); // Pieza 13 (Amarillo)
615 }
616 // =====

```


EJECUCIÓN Versión 2

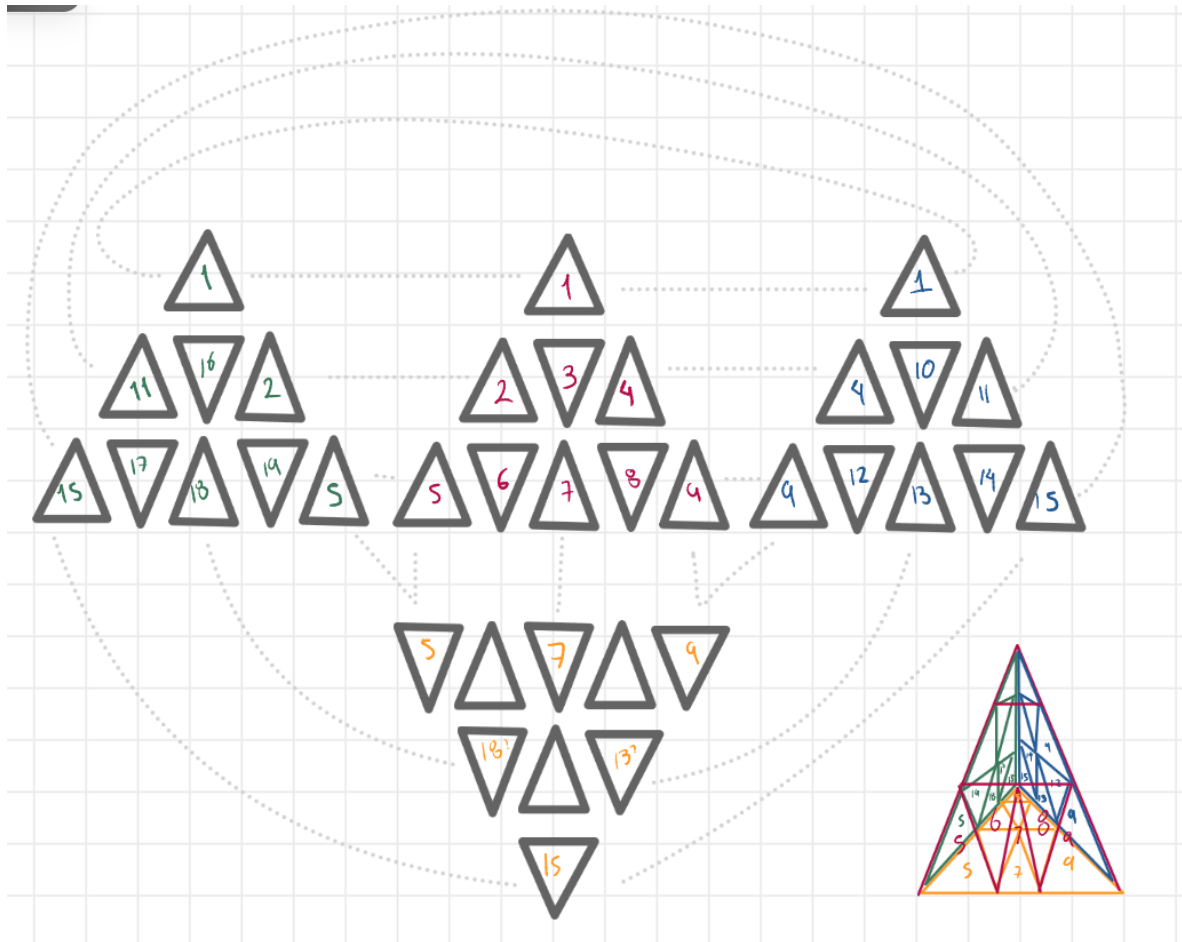
A diferencia de la primera versión, ahora podemos apreciar como no hay separación entre las aristas, por ejemplo, de la pirámide de cualquier esquina, esto ya que estamos trabajando con una sola instancia en esos puntos.



Se adjunta un link a un GIF para visualizar mejor el **pyraminx VERSION 2**

<https://drive.google.com/file/d/15fTDfUyXnlS6c4z9wYoDCA3DRJozQ8V0/view?usp=sharing>

PROBLEMATICAS PRESENTADAS



El definir la piramide rubik fue el primero de los retos, entender su estructura fue tardado, en especial para buscar la versión dos donde se buscaban menos de 36 instancias. En la primera versión muchas, muchas veces, las piramides de desalineaban, no rotaban correctamente y hasta se perdían entre la profundidad del núcleo negro, fue un trabajo arduo solucionarlo después de hacer algunos diagramas. La mayor problemática sin embargo, fue plantear la segunda versión, implementar piezas compuestas fue buena idea, pero no por ello fue más sencillo de entender, aunque gracias a la práctica pasada se pudo solucionar la pregunta de como instanciar y “colorear” de manera diferente cada pequeña pieza del Pyraminx.

CONCLUSIONES

Sobre el ejercicio

En esta práctica, generar la Pyraminx con subdivisión visual de 9 piezas por cara, cuidando que las separaciones fueran distinguibles mediante “líneas” oscuras fue un gran reto a comparación de actividades anteriores. La propuesta 1 nos permitió construir el modelo de manera directa instanciando un total de 36 pirámides (9 por cara) más un núcleo negro, lo cual nos ayudó a facilitar la validación rápida de posiciones y orientación de cada cara. Sin embargo, la propuesta 2 resultó mucho más desafiante a nivel de diseño, ya que buscó una

construcción más cercana a un rompecabezas real, reduciendo el número de instancias a 23 mediante piezas únicas, implementar color por cara usando atributos por vértice.

Comentarios generales

Nuevamente hubo dudas sobre la propuesta dos que pidió el profesor, personalmente no comprendí si quería ambos ejercicios o solo el que saliera mejor en práctica, motivo por el cual adjunto dos versiones.

Cierre

En general, podemos finalizar concluyendo que el ejercicio fue demandante porque requirió combinar correctamente transformaciones (traslación, rotación y escala) más allá de prueba y error, mantener un pequeño control de cámara y recordar el manejo de shaders para lograr una figura coherente en 3D.

BIBLIOGRAFÍA Y REFERENCIAS.

- Khronos Group. (s. f.). *OpenGL Wiki: OpenGL Object* (visión general del pipeline y objetos). OpenGL Wiki. Recuperado el 3 de marzo de 2026, de https://www.khronos.org/opengl/wiki/OpenGL_Object
- Khronos Group. (s. f.). *OpenGL Wiki: Vertex Specification* (VAO/VBO, atributos, layout). OpenGL Wiki. Recuperado el 3 de marzo de 2026, de https://www.khronos.org/opengl/wiki/Vertex_Specification
- Khronos Group. (s. f.). *OpenGL Wiki: Vertex Shader* (cómo funciona el shader de vértices y matrices). OpenGL Wiki. Recuperado el 3 de marzo de 2026, de https://www.khronos.org/opengl/wiki/Vertex_Shader
- Khronos Group. (s. f.). *OpenGL Wiki: glUniform* (envío de uniforms como color y matrices). OpenGL Wiki. Recuperado el 3 de marzo de 2026, de <https://www.khronos.org/opengl/wiki/GLAPI/glUniform>
- LearnOpenGL. (s. f.). *Transformations* (translate/rotate/scale con GLM y modelo-vista-proyección). LearnOpenGL. Recuperado el 3 de marzo de 2026, de <https://learnopengl.com/Getting-started/Transformations>
- LearnOpenGL. (s. f.). *Coordinate Systems* (MVP, cámara y proyección para visualizar bien la escena). LearnOpenGL. Recuperado el 3 de marzo de 2026, de <https://learnopengl.com/Getting-started/Coordinate-Systems>
- LearnOpenGL. (s. f.). *Instancing* (cómo dibujar muchas “piezas” eficientemente y enfoques para instanciar). LearnOpenGL. Recuperado el 3 de marzo de 2026, de <https://learnopengl.com/Advanced-OpenGL/Instancing>
- OpenGL Mathematics (GLM). (s. f.). *GLM manual / documentation* (referencia de glm::mat4, translate, rotate, scale). Recuperado el 3 de marzo de 2026, de <https://glm.g-truc.net/0.9.9/index.html>
- GLFW. (s. f.). *GLFW Documentation* (ventana, input y contexto OpenGL). Recuperado el 3 de marzo de 2026, de <https://www.glfw.org/documentation.html>